Key Lookup versus File Search for a Coding Agent

Retrieval Cost and Reliability of a Curated Plain-Text Index

Vasili Gavrilov
ORCID 0009-0007-9371-5994

November 2026

Abstract

How many tokens does a coding agent spend to find an answer by searching the files, against recalling it from a curated key index? The cost of search grows with the store and with the vagueness of the question, and the cost of a key lookup is the size of what the key returns: the mechanism says so, and one store is measured here. We measure both on one project of about 100k lines of code: an agent loop answers the same questions twice in fresh conversations, once with grep and read, once with recall by exact keys over the author's own index, five repeats each. The ratio depends on what the key returns and who consumes it. When the index holds a note or a reference that the person then opens, the model reads nothing but the note: the retrieval payload is 69 times smaller than search's at the mean and the total bill four times smaller at the median, the index answering every question and search three in four. When the index holds a path that the model is then made to read, both arms pay for the reading and the advantage shrinks to about three and a half times; where a file name already carries the question's words, grep finds it on the first call and the index saves nothing. Run on the command line, the same lookup costs zero model tokens. The limits are stated: the baseline is lexical search, similarity search is not measured, the index carries one user's vocabulary, and the grading favours a curated note by construction.

Keywords: large language model agents; retrieval cost; associative key index; faceted classification; retrieval-augmented generation; code search; Model Context Protocol; reproducible measurement.

1 The question

Consider a photo library. One picture is of *Mary* and *Mother*, at the *beach*, in *Hawaii*, in *2019*, all at once; no folder holds it without forcing one parent and losing the rest. What you want to ask is the combination, every photo of Mary with Mother, and not the videos. Two strategies answer it. *Search* the content: scan the items and match by text or similarity, then filter. Or keep a curated *index*: file each item under one or more keys and recall it by their intersection. The index is an old idea, traceable to Bush's memex [1]; its non-hierarchical, combine-the-facets form is Ranganathan's faceted classification [2], though with free keys combined by intersection it is nearer the tagging that Golder and Huberman describe [21] than to Ranganathan's citation order. Keys are facets, a query is their set intersection (`mary` ∩ `mother` ∩ `photo`); the index is plain text on the user's own machine, and retrieval is a deterministic lookup, not a similarity search. The personal-information-management literature studied the same choice for a person's own files, re-finding by remembered attributes against searching content [19, 20]; the question here is the same choice priced in tokens for a model.

The index measured in this paper, AIS, is a plain-text associative index the author built for exactly this purpose: personal, un-hierarchical material, photos, notes, references, filed under keys and recalled by their intersection. A language model given a large codebase faces the same choice, grep and read its way to an answer, or recall it from such an index, and the cost is simply easier to count there. The two put context into the model but compress it differently. Embedding search compresses the content into a learned embedding space and retrieves by similarity; the key index retrieves by exact address, *compression in keys*. That distinction, and the case for compressing the *access* rather than the stored content, is argued in [14]. The present paper is the *measurement*: it takes that distinction as given and asks the narrow, quantitative question, *for the same question, how many tokens does each spend to find the answer, and how reliable is each?* We answer it on a real project, and report the regime-dependent outcome, including the case where the index does *not* win by much.

2 The two strategies

Content search (lexical or embedding). The alternative to a key index is to search the content itself, lexically (grep) or semantically (an embedding search over chunks, the dense-retrieval [5] core of RAG [4]). Either way the agent scans the corpus and pulls candidates into context. We measure the lexical baseline, the family of which BM25 is the standard form [22]: the agent is given `grep`, `read_file`, and `list_files`, greps for likely terms, reads candidates, and answers. Its cost is the content it pulls in, which grows with the repository and spikes when a common term matches widely. Beyond cost, a large, undifferentiated context also degrades answer accuracy, the "lost in the middle" effect, where a relevant fact buried in a long input is missed [23]; a smaller retrieval payload therefore helps on two axes at once.

Compression in keys (the index). The agent is given the index tools (`recall`, `keys`). Keys are exact: an unknown key returns the empty set, an immediate signal to add or correct a key; a known key returns what was filed under it, and nothing verifies the value. The empty set checks the address, and the key list is the vocabulary. The recalled value is a short note, a URL, or a path; when it is the answer or a reference, the human (or a downstream step) consumes it directly and the model does not expand it.

3 Apparatus

Both the plugin and the harness are the author's software, in the repository cited [15]; both are thin wrappers over a single-binary ANSI C associative-index engine whose store is plain text:

The plugin. A Model Context Protocol (MCP) server [10]. MCP is an open standard for offering a language model a set of callable tools; this server exposes the index's operations, recall by keys, store, list keys, as typed tools to any MCP-capable chat client (terminal agent or desktop), as a local process over standard input and output. It is stateless: the plain-text index remains the source of truth, and the server simply forwards calls to the engine. It optionally logs each call's payload size, not its content, and latency, which is what made the measurement below possible. It is the integration used in practice. The store never leaves the machine; a recalled value does, inside the request to the model, as does every `grep` result in the search arm.

The harness. A *harness*, in software testing, is the small program that runs an experiment automatically: it poses each question to the model, gives it a fixed set of tools, executes whatever

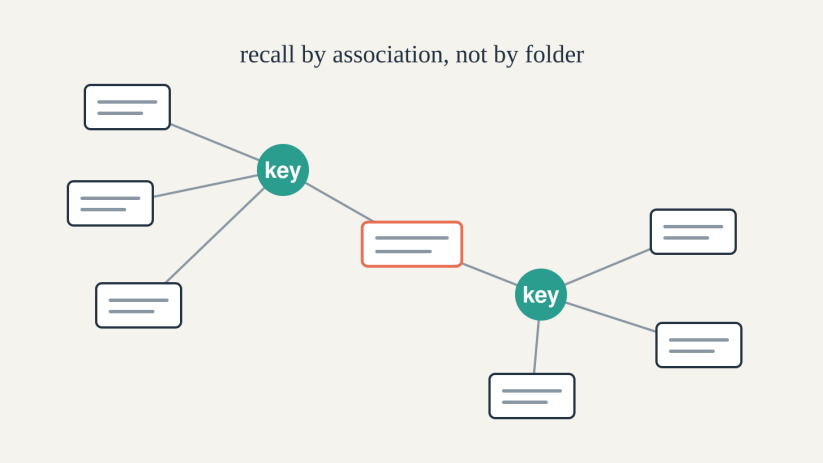


Figure 1: Compression in keys. Each item is filed under one or more keys; items sharing a key are recalled together, and a query is the *intersection* of the lists of items filed under each key (their posting lists), a deterministic lookup on the user's machine.

tool the model asks for, feeds the result back, and records what was spent. Ours is a minimal *agent loop*, the pattern in which a model may call a tool, see the result, and call again until it is ready to answer, driving a commercial model (Claude Sonnet, model id `claude-sonnet-4-6`, June 2026) through the Messages API. For one question it runs:

1. Send the question, a fixed system instruction, and *one* of two tool schemas: the index set, `recall(keys)` and `keys()`, or the file-search set, `grep(pattern)`, `read_file(path)` and `list_files(glob)`.
2. If the model returns a tool call, execute it locally (against the engine, or the file system), append the result to the conversation, and repeat from step 1.
3. If the model returns text and no tool call, that is the final answer; stop.

Execution is local and identical for both arms except the tool set. The harness exposes `recall` and `keys` only; the plugin's `store` is omitted, so the measured index holds only what the user filed. In the main run the index arm has *no* `read_file`: recall returns the answer, and a recalled reference is opened by the human, not the model. Each question is answered in a *fresh* conversation, so runs share no context. Parameters: at most 14 tool rounds per question; each tool result capped at 6,000 characters, about 1,500 tokens; `grep` returns its first 200 matching lines and `list_files` its first 300 paths; `max_tokens` 1,024 per call. Because the harness drives the same recall tool the plugin exposes, what it measures is what production use would pay.

4 Method

What we count. For each question and arm, summed over all turns of the loop, we record:

- *total tokens*: the API's reported input plus output tokens across every turn. This carries a fixed per-turn overhead, the system instruction, the tool schemas (re-sent each turn), and the model's own output, of roughly 2,500 to 3,000 tokens per question for the index arm, whose two calls are its whole transcript; search pays the same preamble on every one of its turns, so

its total grows with its turn count. Cost is reported beside correctness because an agent's token bill is part of its result [17].

- *retrieval payload*: the total size of all *tool results* returned to the model, the content pulled into the context window to find the answer, estimated at four bytes per token (total tokens are the API's counts; the payload is not). This is the quantity the index optimizes, and unlike total tokens it is not diluted by the fixed overhead. It is the headline metric.
- *correctness*: whether the expected answer string appears in the final response; for set questions, *item-recall*, the fraction of expected items present.
- tool rounds and wall-clock latency.

Controls. Both arms answer the same questions, with the same model, in fresh conversations. The index directory is excluded from the file-search arm, so it cannot grep the index itself and must find answers in the genuine project files. Tool outputs are capped (grep to its first matches, reads to a fixed size) so no single call can dump the whole repository; this bounds the search arm *in its favour*, the uncapped gap would be larger. The main run and the path-with-read run are each 8 questions over 5 repeats; the single-value toy was one pass, treated as illustrative. The searched tree held source and documentation; the index content was not duplicated in it.

Project. The search target is a real product repository. The search arm greps the working tree the agent actually sees: about 0.35 GB across roughly 1,300 files and 100k lines of source in June 2026, and 3,323 files at 0.21 GB when the path-with-read run was made in November, the same repository four months on with its private notes excluded and its source lines not recounted. (The June checkout was 1.9 GB on disk, but 1.6 GB of that is version-control history under `.git`, which is excluded and never searched.) An *uncapped* grep for a single common term over this tree would return on the order of 17,000 to 110,000 tokens of raw matches; the harness caps each tool result to about 1,500 tokens, so the search agent pays this haystack across many capped turns rather than in one dump.

What the index stores sets the regime. We vary it deliberately: *paths* (the key maps to a document path, and the agent reads the document to answer) versus *notes and references* (the key maps to a short note or a URL that *is* the answer, recalled and returned, no reading). Questions are natural-language and varied ("how do I deploy a release to dev", "give me all the documents about deployment").

The runs. We iterated. A single-fact toy (questions returning one short value) was trivial and is not tabulated. A June document-set run in which the index returned a path *and the agent was made to read the document* charged the model for reading, which is not how the tool is used, since the person opens a recalled URL or file; it was one pass and is not reported. The **main run** stores notes and references that are themselves the answer and has the index arm *recall only*, no reading. That configuration was fixed before its five repeats were run, and Section 5 reports it. The **path-with-read run** of 2026-11-02 replaces the June pass: 8 questions, 5 repeats, same model and harness, a nine-record index of paths to the project's deployment documents, its environment list and its release script, and the main run's questions with dev read as uat; it is the second row of Table 2 and the paragraph after it.

5 Results

The main run (notes/references, no model reading), 8 questions × 5 repeats. The index recalled about **68 tokens** of content per question; file search pulled in about **4,744**, a **69×** retrieval-payload ratio (ratio of the means over the 40 runs, 68.9). The index answered **40 of 40** with a tight spread (mean about 2,900 tokens, standard deviation 518); file search answered **31 of 40** with enormous variance (standard deviation about 36,000, larger than its own mean). Item-recall, the fraction of expected items present on the set questions, coincides with the hit rate (1.0 and 0.78): every search miss was a complete miss. Table 1 and Figure 2 give the per-question means. The failures are not confined to the expensive runs: search missed `release_packaging` on *all five* repeats (at 6,400 to 72,000 tokens), yet usually *found* `monitoring` despite spending 17,000 to 170,000 tokens to do so. Both documents exist in the project; search fails to find `release_packaging` within the turn budget, not for its absence. Cost and correctness fail independently.

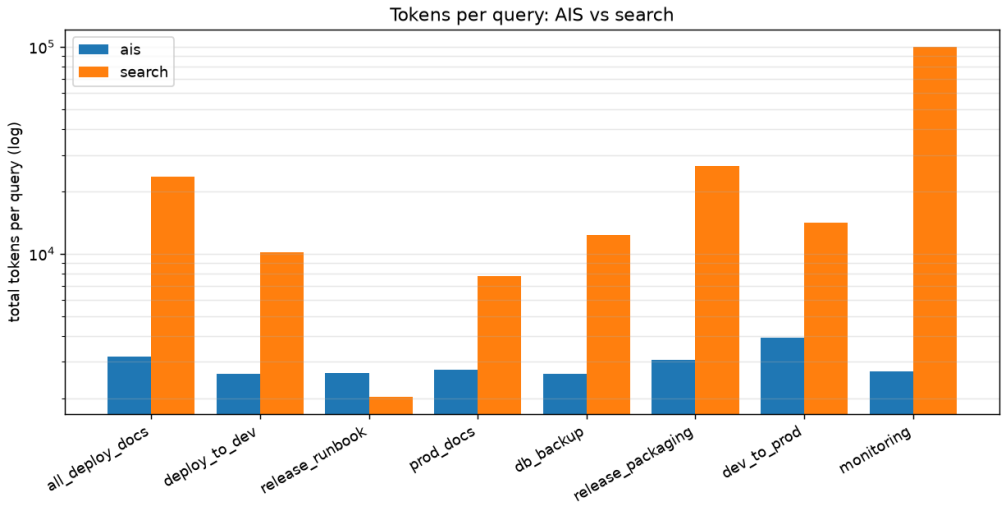


Figure 2: Per-question total tokens (log scale), main run. The index (left bars) stays low and flat; file search (right bars) is higher and spikes on common-term greps (*release packaging*, *monitoring*).

question	index tok	file-search tok	ratio
`all_deploy_docs`	3,180	23,611	7×
`deploy_to_dev`	2,635	10,173	4×
`release_runbook`	2,663	2,039	1×
`prod_docs`	2,745	7,768	3×
`db_backup`	2,632	12,372	5×
`release_packaging`	3,059	26,438	9× (search miss)
`dev_to_prod`	3,931	14,066	4×
`monitoring`	2,706	99,499	37×
mean	**2,944**	**24,496**	

Table 1: Per-question total tokens, main run (note/reference index, no model reads). Common-term greps over the searched tree are where search explodes (*monitoring*) and where it misses (*release packaging*); the two failures are independent.

Why the total-token ratio understates it. Read on *total* tokens, the index beats search by a few times: the ratio of the arms' medians is 4.3× and of their means 8.3×. That total is diluted: both arms pay the same preamble on every turn (system prompt, tool schemas re-sent each turn, the model's prose answer), and the index arm's whole transcript is two calls. The retrieval payload is 19 per cent of search's bill; the rest is that preamble and the growing transcript re-sent over search's 2 to 14 tool calls per question, against the index's 2 to 3. The payload (Table 2) isolates the quantity the index optimizes; the turn count is why search's total is larger still.

index returns	**total ratio**		**retrieval ratio**	**correct, of 40**	
	median	mean		index	search
note / URL, no read (main run)	**4.3×**	**8.3×**	**69×**	**40**	31
path, the model reads it (path-with-read run)	3.4×	6.6×	3.5×	40	35
direct CLI, no model	0 tokens		n/a	exact	n/a

Table 2: Regimes. Both measured rows are 8 questions over 5 repeats; the total ratios are of the arms' medians and means, the retrieval ratio of the arms' mean payloads, and the last two columns are correct answers out of 40 per arm. Direct CLI is the zero-token path. The single-value toy was one pass, not tabulated.

The path-with-read contrast. When the index returns a document path and the agent then reads it, both arms pay for the reading; the index saves only the *finding*, and the advantage shrinks to 3.5× on payload. The index arm took three calls on every question (six for the one whose answer is four documents), a median of 6,215 total tokens and a mean of 6,878; search took 3 to 24 calls, a median of 21,020 and a mean of 45,175, one run reaching 277,000. The per-question ratio of medians runs from 0.6× to 31×. Where a file name carries the question's words (`release_runbook`, `db_backup`) grep finds it on the first call, both arms take three, and the ratio is 1.1× to 1.3×; for the four-document question the index arm reads four files and costs more than search's three to four calls from one directory listing. The tree differs from the main run's (Section 4); it moved search's payload from 4,744 to 7,133 tokens and the index arm's from 68 to 2,047, so the fall in the ratio is the reading. The literal grader failed five search answers. Read by the author (the answers are not deposited, Section 9): two name the Ant target and the zip without either graded string, so search is 37 of 40 by hand and the index arm, read the same way, stays 40; two lists each omit one of the three graded documents; one run used its fourteen rounds (21 calls) without answering. This is the avoidable case: do not make the model expand a recalled reference.

Direct invocation. For a known, repeated key the user (or a script) runs the lookup on the command line and the model is never involved: zero model tokens. The agent comparison cannot show this, because it forces every query through the model; it is the ceiling of the economy.

6 Analysis

The regime is a design choice. Same project, parallel questions, same harness and model; the index held notes in one run and paths in the other. What moved the retrieval advantage between the two regimes of Table 2 was *what the key returns and who consumes it.* Store a path and make the model expand it, and you pay for reading, which search pays too. Store the answer (a note) or a reference (a URL) and let the person open it, and retrieval is close to free. Paths answer "*which* document"; notes answer "*what* is the answer".

Cost versus repository size. One repository was measured, so this paragraph argues from

the mechanism. The index's retrieval cost is constant in repository size by construction (a key addresses an index entry), while file search grows with the corpus and spikes on common terms: the `monitoring` question of Table 1 averaged about 99,000 total tokens across repeats. These totals are dominated by the transcript re-sent over 16 to 20 turns; the retrieval payload on those queries is about 15,000 to 17,000 tokens. As a project grows the gap should widen; measuring it needs a second repository or nested subsets of this one.

Reliability. The index was exact on every question and every repeat; file search was wrong on 9 of 40 runs and failed `release_packaging` on all five (Section 5). This follows from the mechanics. Exact-key lookup returns either what was filed or the empty set, a detectable miss; a wrong or stale note under a known key returns as confidently as a right one. A `grep` miss is detectable too; a `grep` hit is a candidate the model must still judge, and the answer it generates from candidates is not guaranteed to signal an error.

7 Estimated savings

The measured per-query figures support a rough projection. For a curated, repeated lookup the saving is about $C_{\text{search}} - C_{\text{AIS}}$, where C_{search} is what the agent spends finding the answer by file search and C_{AIS} is what the index spends. From the main run, $C_{\text{AIS}} \approx 2{,}900$ tokens through the model, or 0 used directly on the command line; C_{search} had a median of 11,500 and a mean of 24,500 tokens over the 40 runs, and grows with repository size by construction. So each curated query saves on the order of 8,500 tokens through the model, and nearly all of C_{search} when the lookup is run directly.

lookups/day	**via model (saves ∼8.5k)**	**direct CLI (saves ∼11.5k)**
30	∼0.25M tok/day	∼0.35M tok/day
100	∼0.85M tok/day	∼1.2M tok/day

Table 3: Projected daily token saving per developer for curated, repeated lookups. The 30 to 100 range is a conservative, human-scale floor: Sadowski et al. [3] report a mean of 12 code-search queries per developer per weekday, median 6, and counting docs, configs, and references adds more. The *agentic* regime this paper targets runs far higher, one task issues many retrievals and a developer may run several agents, so the saving grows rather than shrinks. We know of no published per-day agent-lookup rate yet, so this is a structural estimate, not a measured deployment; quantifying real lookup volume, human and agent, is a natural next step. Over about 20 working days the range is roughly 5 to 20 million tokens per developer per month, which at the Sonnet 5 list price of 2 USD per million input tokens [12] is 10 to 40 USD per heavy user (mostly input tokens). The projection assumes every lookup is a curated hit; a miss costs the recall round and then the search.

Three qualifications. The saving applies only to *curated, repeated* lookups, the value regime; first-time or unknown-key questions fall back to search and save nothing (the empty result instead tells you what to add). C_{search} is noisy, so the claim is "one to two orders of magnitude per curated query", not a point estimate. And the largest lever is direct use: a lookup run on the command line costs zero model tokens (Section 5).

8 Threats to validity

These are small pilots: 8 questions over 5 repeats, on one project, with one model. The search arm is noisy across repeats (its standard deviation exceeds its mean), so we report its central tendency as a median and its reliability as a rate over all 40 runs. Grading is a substring match on the final answer, which understates retrieval success on "how-to" questions where the right document was found but its filename was not echoed. Conversely it favours the index, whose curated note contains the graded substring by construction, so the correctness gap reflects curation as much as retrieval mechanics. The index is curated, so it answers only what was filed; un-curated or unknown-key questions fall back to search (itself informative: the empty result tells you what to add). The file-search agent is a basic grep-and-read loop with capped tool output, the same shape as the agent-computer interface of SWE-agent [18], not a tuned embedding pipeline; a better one would change the constants but not the asymptotic shape (search cost grows with the corpus; key lookup does not). A note is data the model reads, with no more authority than a file: if the model may `store`, or the index is shared or synced, a note can carry instructions, and a recalled URL is a stored pointer trusted exactly as much as its author, so it should not be fetched automatically. The measurement assumes a single human author and a read-only agent. The numbers illustrate the regimes; they are not a benchmark.

9 Reproducibility

The target project and its index are private and are not released. Deposited with this paper, in `ais/experiment/` of the articles repository (`https://github.com/Anode1/articles`): the harness, the table generator, the main run's sanitized results CSV, and a public stand-in corpus, index and question set (a small photo library) that reproduces the method on any machine with the engine [15] built. The main run is the note/reference, no-read configuration, `run.py --ais-no-read --repeats 5`, over 8 questions against the private project; re-running it needs a project and an index of one's own and yields different constants. `analyze.py` emits Table 1 *directly from the deposited CSV*, with no API key, so the printed numbers cannot drift from the data. The path-with-read run is `ais/experiment/pathread/` there: its questions, the script that builds its nine-record path index, the invocation (`run.py --exclude PRIVATE` keeps the project's private notes out of both arms) and its sanitized CSV; its questions are deposited, since their expected strings name only tool and file names, and its answers are not, since they carry the project's paths. The CSV is stripped of the free-text answer column (which embeds project-internal paths and names) and keeps, for every question and repeat, the per-arm total, retrieval, input and output tokens, tool calls, latency, correctness, and item-recall. The model is `claude-sonnet-4-6` through the Messages API, the main run in June 2026 and the path-with-read run on 2026-11-02. Absolute token counts move with model version; the regime structure follows from the mechanism, search cost growing with the corpus and key lookup staying flat. The text was drafted by the author and edited with Claude; the author takes responsibility for its contents. The model appears in the measurement only as the system under test.

10 Related work

The direction measured here is, as of 2026, an active convergence. Structured, tool-mediated retrieval is reported to beat vector search on token cost: a tree-sitter knowledge graph exposed over MCP shows about ten times lower token cost and roughly half the tool calls of iterative file exploration,

at a modest quality cost (83% versus 92% answer quality) [7], and agentic keyword search reaches RAG-level performance with no vector store at all [6]. A unified review of *externalization* in LLM agents, memory, skills, protocols, and harness engineering, maps the area [9], while efficiency- and interpretability-oriented memory frameworks pursue the same end over latent graphs [8]. The index measured here descends from the author's own index work, begun as a by-hand filesystem index about 2001 and registered on SourceForge in November 2004 [16], rather than from this 2026 convergence.

The systems above are *auto-extracted* (parsed code graphs, embeddings) and *similarity-based*. The index measured here is *exact* and *human-curated*: the keys are the user's own prior, recall is a deterministic lookup, and a miss returns the empty set. The store is plain text on the user's machine, and only a recalled value travels to the model. And we isolate the regime the agentic-retrieval literature routes past, direct lookup with the model out of the loop. The measurement here gives the cost and the miss rate of one exact, curated index against one lexical search, on one project.

11 Discussion

The floor is reducible

The fixed per-turn overhead is not fundamental. It is the system prompt and tool schemas re-sent each turn; it dilutes the total-token ratio, and it let file search win the one easy, unique-key question, whose lighter preamble cost less over two equally cheap calls. It is reducible three ways that compound. *Prompt caching* bills a marked, re-sent prefix above a minimum length at one tenth of the base input price for the model used, within the cache lifetime, after a first call charged at 1.25 times base [11, 12]; OpenAI discounts a cached prefix by up to 95 per cent [13]. That prefix is the system prompt, the tool schemas, the user's standing instructions, the prior itself, and the conversation re-sent each turn. Stripping it from *both* arms leaves each query's cost at the *fresh* content it pulls in, tens of tokens for the index against thousands for file search, so caching favours neither arm: it collapses both totals toward the retrieval payload. A *minimal recall-only tool surface* (one `recall(keys)` tool) shrinks the preamble even uncached, and removes the case where the index carries a heavier preamble than search. And *recall-first routing*, try the exact lookup first and let the empty set, the checksum, trigger the fall-back to file search, pays the cheap path on every curated hit and the expensive one only on a genuine miss, with the exactness telling you which. Together these push the via-model regime toward the zero-token direct-use floor, taking retrieval off the model on the agent path too. We did not implement them in the harness; they are the natural next step the measurement points to.

Table 4 sketches the effect on per-query cost.

An untested property

A property independent of token cost follows from who authors the keys. Embedding retrieval returns what is typical in the model's training distribution, an average over many corpora; the curated index returns what *this* user filed, in *this* user's vocabulary, with no regression toward the conventional when the task is novel. Nothing in the measurement above tests this.

approach (curated lookup)	model tokens / query
file search (no index)	∼11,500 (to ∼170,000)
index via model, current	∼2,900
index via model, cached preamble + minimal surface	∼600 *(projected)*
index, direct CLI (no model)	0

Table 4: Per-query cost for a curated lookup, current and projected. Prompt caching bills a re-sent prefix at one tenth of base on the endpoint used here, after a first call at 1.25 times base, and the projection assumes the prefix stays cacheable and warm; a minimal recall-only surface shrinks the prior further, so the index's via-model cost falls to a few hundred tokens, within reach of the zero-token direct floor. Caching reduces file search too, but only its re-sent overhead; what remains is its fresh retrieval content (thousands of tokens), so the gap persists. The file-search figure is the measured uncached baseline; the projected index row is not implemented in our harness.

12 Conclusion

The token economy of a curated key index over an agent's file search is real but conditional, and the condition is in the engineer's hands. Index the answer or a reference and let the human consume it, and retrieval costs almost nothing: zero tokens directly, about 70 times less retrieval payload through the model and 4 to 9 times fewer total tokens; by construction the cost does not grow with the project, and the lookup returns what was filed. Make the model expand a recalled document instead, and the advantage shrinks to a few times. The rule is short: compress the access, not the store. Return the answer or a pointer, and keep retrieval off the model wherever a lookup will do.

Acknowledgements

The index-as-graph and faceted-key model used here has its roots in the advanced graph-theory course of the late Professor Shimon Even (1935–2004), whose network-flow algorithms reach a global result through local rules, and whose student the author had the privilege to be.

References

[1] V. Bush. *As We May Think.* The Atlantic, 1945.

[2] S. R. Ranganathan. *Colon Classification.* Madras Library Association, 1933.

[3] C. Sadowski, K. T. Stolee, S. Elbaum. *How Developers Search for Code: A Case Study.* FSE 2015.

[4] P. Lewis, E. Perez, A. Piktus, et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* NeurIPS 2020; arXiv:2005.11401.

[5] V. Karpukhin, B. Oğuz, S. Min, et al. *Dense Passage Retrieval for Open-Domain Question Answering.* EMNLP 2020; arXiv:2004.04906.

[6] S. Subramanian, A. Akinfaderin, Y. Zhang, et al. *Keyword Search Is All You Need: Achieving RAG-Level Performance without Vector Databases Using Agentic Tool Use.* AAAI 2026; arXiv:2602.23368.

[7] M. Vogel, F. Meyer-Eschenbach, S. Kohler, E. Grünewald, F. Balzer. *Codebase-Memory: Tree-Sitter-Based Knowledge Graphs for LLM Code Exploration via MCP.* arXiv:2603.27277, 2026.

[8] X. Zhang, K. Yang, H. Li, C. Li, Q. Wei, S. Ananiadou. *Implicit Graph, Explicit Retrieval: Towards Efficient and Interpretable Long-horizon Memory for Large Language Models.* arXiv:2601.03417, 2026.

[9] C. Zhou, H. Chai, W. Chen, et al. *Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocols and Harness Engineering.* arXiv:2604.08224, 2026.

[10] *Model Context Protocol* specification. Introduced by Anthropic, 2024; a project of the Agentic AI Foundation (Linux Foundation) since December 2025. modelcontextprotocol.io, accessed 2026-11-02.

[11] Anthropic. *Prompt Caching.* platform.claude.com/docs/en/build-with-claude/prompt-caching, accessed 2026-11-02.

[12] Anthropic. *Pricing.* platform.claude.com/docs/en/about-claude/pricing, accessed 2026-11-02.

[13] OpenAI. *Prompt Caching.* developers.openai.com/api/docs/guides/prompt-caching, accessed 2026-11-02.

[14] V. Gavrilov. *Intelligence Is the Discovery of Compressors.* Zenodo, 2026. DOI: 10.5281/zenodo.20440110.

[15] V. Gavrilov. *AIS*: associative-indexing engine and MCP plugin. github.com/Anode1/ais, 2026.

[16] V. Gavrilov. *AIS: Priority and Provenance Record.* Zenodo, 2026. DOI: 10.5281/zenodo.20647047.

[17] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Resnik, A. Narayanan. *AI Agents That Matter.* arXiv:2407.01502, 2024.

[18] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, O. Press. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering.* NeurIPS 2024; arXiv:2405.15793.

[19] S. Dumais, E. Cutrell, J. J. Cadiz, G. Jancke, R. Sarin, D. C. Robbins. *Stuff I've Seen: A System for Personal Information Retrieval and Re-Use.* SIGIR 2003.

[20] W. Jones. *Keeping Found Things Found: The Study and Practice of Personal Information Management.* Morgan Kaufmann, 2007.

[21] S. A. Golder, B. A. Huberman. *Usage Patterns of Collaborative Tagging Systems.* Journal of Information Science 32(2), 2006.

[22] S. Robertson, H. Zaragoza. *The Probabilistic Relevance Framework: BM25 and Beyond.* Foundations and Trends in Information Retrieval 3(4), 2009.

[23] N. F. Liu, K. Lin, J. Hewitt, et al. *Lost in the Middle: How Language Models Use Long Contexts.* TACL 2024; arXiv:2307.03172.